

React 的 `useState`、`useRef` 和 `useEffect` hooks 的影片播放器組件。

讓我們詳細解釋程式碼中的不同部分：

引入 React 和相關 hooks：首先，我們使用 `import` 語句導入了 `React`、`useState`、`useRef` 和 `useEffect` 這些需要使用的 React hooks。

定義 `VideoPlayer` 組件：我們定義了一個名為 `VideoPlayer` 的 React 組件。

使用 `useRef` hook 建立 `videoRef` 參考：我們使用 `useRef(null)` 建了一個 `videoRef` 參考，並將其初始值設為 `null`。這個引用將用於訪問 `<video>` 元素。

使用 `useState` hook 建立 `currentTime` 狀態：我們使用 `useState(0)` 建了一個 `currentTime` 狀態變數和一個 `setCurrentTime` 函數，初始值設為 `0`。這個狀態將用於存儲當前的影片時間。

使用 `useEffect` hook 設置計時器：我們使用 `useEffect` hook 來設置計時器。在組件首次渲染時，這段代碼將運行一次。計時器每秒執行一次，檢查 `videoRef.current` 是否存在並檢索 `currentTime`，然後使用 `setCurrentTime` 函數將當前時間更新到狀態變數中。

清理計時器：為了避免內存洩漏，我們使用 `return` 語句在 `useEffect` hook 中返回一個清理函數。這個清理函數將在組件卸載之前執行，它清除了設置的計時器。

返回 JSX：在 `render` 方法中，我們返回一個包含 `<video>` 元素和顯示當前時間的 `<div>` 元素的 JSX。

```
import React, { useState, useRef, useEffect } from 'react';
```

```
function VideoPlayer() {  
  const videoRef = useRef(null);  
  const [currentTime, setCurrentTime] = useState(0);  
  
  useEffect(() => {  
    const interval = setInterval(() => {  
      if (videoRef.current && videoRef.current.currentTime) {  
        setCurrentTime(videoRef.current.currentTime);  
      }  
    }, 1000);  
  });  
}
```

```

        return () => {
            clearInterval(interval);
        };
    }, []);

    return (
        <div>
            <video ref={videoRef} src="startrek.mp4" controls />
            <div>Current Time: {currentTime}</div>
        </div>
    );
}

```

```
export default VideoPlayer;
```

焦點切換

假設你正在開發一個多步驟表單，每個步驟都有多個輸入欄位，你希望在用戶完成當前步驟時自動將焦點設置在下一個步驟的第一個輸入欄位上。你可以使用 `useRef` 來存儲下一個步驟的第一個輸入欄位的引用，並在當前步驟完成時使用 `nextStepRef.current.focus()` 方法來設置焦點。

```
import React, { useRef } from 'react';
```

```

function NextStep() {
    const nextStepRef = useRef(null);

    const handleCurrentStepCompletion = () => {
        // 當前步驟完成後，設置焦點到下一步驟的第一個輸入欄位
        nextStepRef.current.focus();
    };

    return (
        <div>
            <input type="text" /><br />
            <button onClick={handleCurrentStepCompletion}>Complete
Step</button>
            <br />

```

```
        { /* 下一步驟的第一個輸入欄位 */  
          <input ref={nextStepRef} type="text" />  
        </div>  
      );  
    }  
    export default NextStep;
```